

SISTEMAS OPERATIVOS

Conferencia 7 · Tema V: Memoria Virtual

Carrera: Ingeniería Informática	Profesora: Ing. Claudia Vecino Fernández
Año: 3ro Período: 2do Tipo: CRD	Correo: claudiavf@uho.edu.cu
Disciplina: Infraestructura de Sistemas Informáticos	Teléfono: 54196249

Objetivo: Comprender el concepto de memoria virtual, el mecanismo de demanda de página, los algoritmos de reemplazo de páginas y sus fundamentos teóricos, el fenómeno de thrashing y los mecanismos para prevenirlo, así como el modelo de working set y las políticas de asignación de marcos. Analizar la administración de memoria virtual en sistemas operativos actuales.

Sumario

- Concepto de memoria virtual. Motivación y beneficios.
- Demanda de página (Page Fault). Flujo completo del tratamiento.
- Rendimiento de la memoria virtual. Cálculo del tiempo de acceso efectivo.
- Prepaginado. Política de tamaño de página.
- Algoritmos de reemplazo de páginas: OPT, FIFO, LRU, Segunda oportunidad, NRU, LFU/MFU.
- Comparativa y evaluación de algoritmos mediante cadenas de referencia.
- Asignación de marcos: política local vs. global.
- Thrashing: causas, detección y soluciones.
- Modelo de Working Set. Frecuencia de fallos de página.
- Overlays: carga dinámica manual (contexto histórico).
- Memoria virtual en SO actuales: Linux (demand paging, OOM killer), Windows.

Bibliografía

- Tanenbaum, A. S. Sistemas Operativos Modernos, 4ª ed. Págs. 123–200.
- Silberschatz, Galvin, Gagne. Operating System Concepts, 10ª ed. Cap. 10.
- Love, R. Linux Kernel Development, 3ª ed. Caps. 11–12.
- Gorman, M. Understanding the Linux Virtual Memory Manager. Prentice Hall, 2004.

1. Concepto de Memoria Virtual

Los sistemas operativos modernos enfrentan dos problemas relacionados con la memoria: los programas pueden ser más grandes que la RAM disponible, y la eficiencia del CPU aumenta con el grado de multiprogramación, que a su vez está limitado por la RAM disponible para almacenar procesos.

Definición: La memoria virtual es una técnica que permite ejecutar procesos que pueden no estar completamente cargados en memoria RAM. El SO mantiene en disco las partes del proceso no activas y las carga bajo demanda, creando la ilusión de que cada proceso dispone de un espacio de memoria mucho mayor que la RAM física instalada.

1.1 Separación entre Espacio Lógico y Espacio Físico

Sin memoria virtual: Proceso $\leq$ RAM física RAM 16 GB Proceso A máx ~16 GB Limitación: multiprogramación de grado bajo.	Con memoria virtual: Proceso puede ser $\gg$ RAM física Espacio virtual 256 TB (x86-64) Proceso A: 4 GB lógicos → solo 50 MB activos en RAM → resto en disco (swap/page) Ventaja: N procesos independientes del tamaño de RAM física.
--	--

La forma más común de instrumentar la memoria virtual es mediante paginación: parte de las páginas del proceso residen en RAM (marcos activos) y el resto en un dispositivo de almacenamiento (área de swap en disco o archivo de paginación).

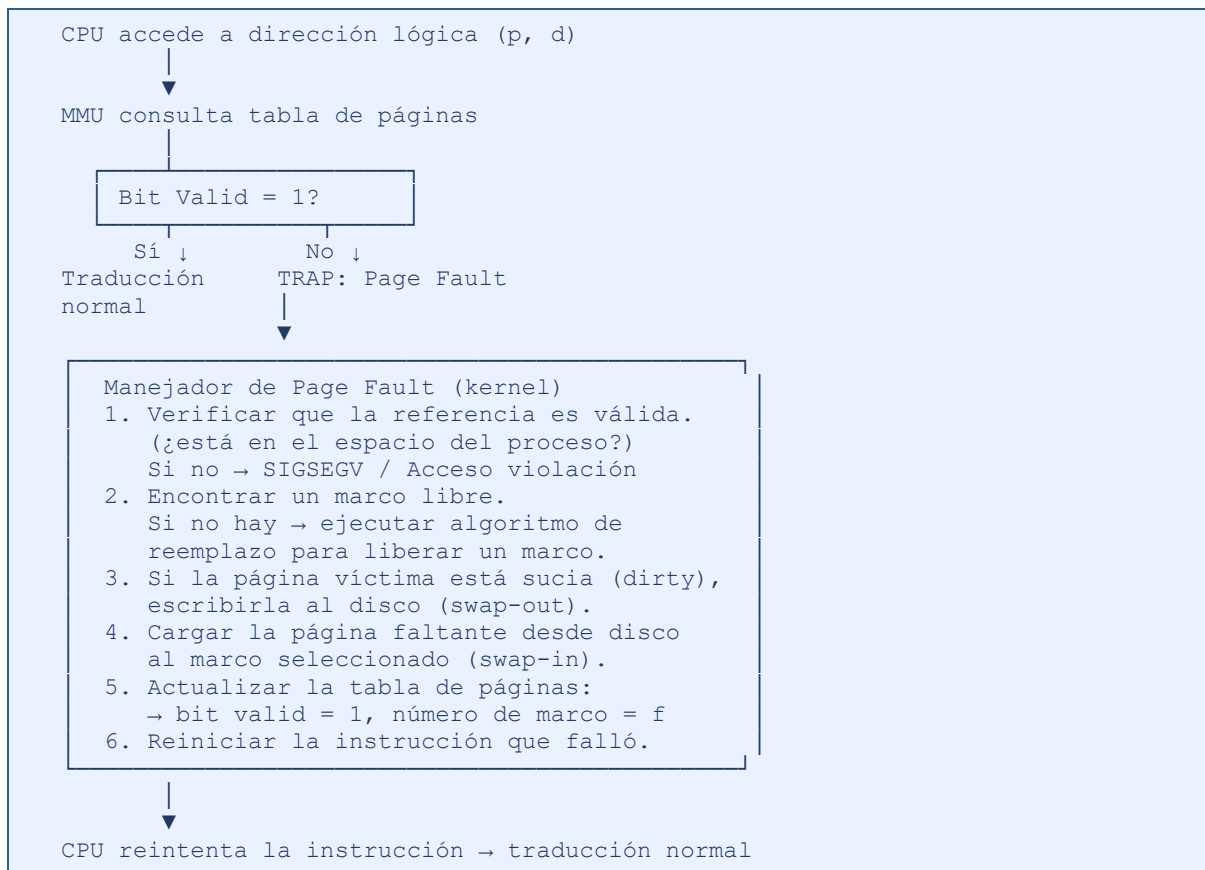
Beneficio	Descripción	Impacto práctico
Programas más grandes	Los procesos pueden superar la RAM física.	Aplicaciones CAD, bases de datos, IDEs sin restricción de tamaño.
Mayor multiprogramación	Más procesos en memoria simultáneamente.	Mejor utilización del CPU; menos tiempo ocioso.
Compartición eficiente	Páginas compartidas (libc, código del kernel).	Ahorro de RAM en sistemas con muchos procesos.
Carga parcial	Solo se carga el código realmente ejecutado.	Arranque más rápido; menor memoria inicial requerida.

✚ **Nota actualizada:** En Linux con x86-64, el espacio de direcciones virtual por proceso es de 128 TB (48 bits de dirección). Android gestiona la memoria virtual de forma agresiva: las aplicaciones en segundo plano pueden tener sus páginas en zswap (comprimidas en RAM) o en el archivo de paginación en flash, siendo restauradas transparentemente cuando el usuario vuelve a la aplicación.

2. Demanda de Página (Page Fault)

Cuando se referencia una dirección cuya página está en RAM, la MMU la traduce normalmente. La complicación surge cuando la página referenciada no está en RAM: el bit de validez (present/absent) en la entrada de la tabla de páginas está a 0, lo que provoca una trampa llamada fallo de página (page fault).

2.1 Flujo Completo del Tratamiento de un Page Fault



2.2 Rendimiento: Tiempo de Acceso Efectivo (TAE)

El page fault implica acceder al disco, lo que es órdenes de magnitud más lento que la RAM. El tiempo de acceso efectivo se calcula como:

$$TAE = (1 - p) \times t_{\text{acceso_RAM}} + p \times t_{\text{page_fault}}$$

Donde:

p = tasa de fallos de página (page fault rate)
 $t_{\text{acceso_RAM}}$ $\approx$ 80 ns (DRAM moderna)
 $t_{\text{page_fault}}$ $\approx$ 8 ms (disco HDD) / 0,1 ms (SSD NVMe)

Ejemplo con HDD ($t_{\text{pf}} = 8 \text{ ms} = 8.000.000 \text{ ns}$):

Si $p = 0,001$ (1 fallo por cada 1000 accesos):

$$TAE = 0,999 \times 80 + 0,001 \times 8.000.000$$

$$= 79,92 + 8.000 \approx 8.080 \text{ ns} \leftarrow 100\times \text{ más lento}$$

Para degradación $< 10\%$: $TAE \leq 1,1 \times t_{\text{RAM}} = 88 \text{ ns}$

$$\rightarrow p \leq (88 - 80) / 8.000.000 \approx 0,000001 \quad (1 \text{ fallo} / \text{millón de accesos})$$

● **Atención:** Un page fault rate de tan solo 1 en 1000 accesos degrada el rendimiento ~100 veces con HDD. Esto muestra la crítica importancia de los algoritmos de reemplazo: deben mantener en RAM las páginas con mayor probabilidad de uso futuro. Con SSD NVMe ($t_{\text{pf}} \approx 0,1 \text{ ms}$), la penalización es ~1250× menor, pero sigue siendo significativa.

2.3 Prepaginado vs. Demanda Pura

Estrategia	Descripción	Ventaja / Desventaja
Demanda pura (Pure demand paging)	Solo se carga una página cuando se produce un fallo. Ninguna página se precarga al inicio.	Minimiza la memoria usada. Muchos fallos al inicio (cold start). Base de todos los SO modernos.
Prepaginado (Prepaging)	Se cargan páginas adicionales que se predicen necesarias pronto, basándose en localidad o historial.	Reduce fallos en ráfagas de acceso. Si la predicción falla, se desperdicia RAM y ancho de banda.
Carga anticipada al fork()	Al crear un proceso hijo con fork(), se usa Copy-on-Write: las páginas se comparten hasta que el hijo escribe. Solo se copian las páginas modificadas.	Muy eficiente: fork() + exec() (el caso común) no copia casi nada.

 **Concepto clave:** Principio de Localidad: los programas exhiben localidad temporal (lo que se usó recientemente se volverá a usar) y localidad espacial (lo que está cerca de lo que se usó recientemente se usará pronto). Esta propiedad fundamental hace posible que la memoria virtual funcione eficientemente: basta mantener en RAM el working set del proceso.

2.4 Tamaño de Página: Compromisos

Aspecto	Páginas pequeñas (1–4 KB)	Páginas grandes (2 MB – 1 GB)
Fragmentación interna	Mínima (último frame casi lleno).	Significativa (puede desperdiciar MB).
Tamaño tabla de páginas	Grande (más entradas).	Pequeña (menos entradas).
Tasa de page faults	Mayor (más fallos, transferencias más pequeñas).	Menor (menos fallos, transferencias mayores).
Eficiencia de E/S	Mala (muchas operaciones de disco).	Buena (pocas E/S grandes son eficientes).
TLB coverage	Baja (cada entrada cubre poco).	Alta (cada entrada cubre 2 MB con huge pages).
Uso en SO actuales	4 KB (página estándar en x86, ARM).	2 MB / 1 GB: Huge Pages para BBDD, HPC.

3. Algoritmos de Reemplazo de Páginas

Cuando se produce un page fault y no hay marcos libres, el SO debe seleccionar una página víctima para reemplazar. El objetivo es minimizar la tasa de fallos de página futura. Se usará la siguiente cadena de referencia para comparar los algoritmos:

 **Ejemplo:** Cadena de referencia: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
Número de marcos disponibles: 3

3.1 OPT — Algoritmo Óptimo (Bélády)

Reemplaza la página que no será usada durante el mayor tiempo en el futuro. Es el óptimo teórico: minimiza la tasa de fallos. Es imposible de implementar en la práctica (requiere conocer el futuro), pero sirve como referencia de comparación.

```
Cadena: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Marcos:
[7][-][-] [7][0][-] [7][0][1] [2][0][1] [2][0][1] [2][0][3]...
F      F      F      F      F
```

OPT produce el número mínimo teórico de fallos de página.
Con 3 marcos y esta cadena: 9 fallos (referencia ideal).

3.2 FIFO — First In, First Out

La página víctima es la que lleva más tiempo en memoria. Simple de implementar con una cola circular. No considera si la página sigue siendo usada activamente.

Marcos (3):	Referencia	Marco 1	Marco 2	Marco 3	Fallo
	7	7	-	-	X
	0	7	0	-	X
	1	7	0	1	X
	2	2	0	1	X (saca 7)
	0	2	0	1	✓
	3	2	3	1	X (saca 0)
	0	2	3	0	X (saca 1)
	4	4	3	0	X (saca 2)
	...	...	...	...	...
Total FIFO: 15 fallos (vs. 9 de OPT)					

⚠ Importante: Anomalía de Bélády (FIFO): al aumentar el número de marcos, la cantidad de fallos de página puede AUMENTAR con FIFO. Este comportamiento contraintuitivo ocurre porque FIFO no es un algoritmo de pila (stack algorithm). LRU y OPT sí son algoritmos de pila: más marcos nunca producen más fallos.

3.3 LRU — Least Recently Used

Reemplaza la página que no ha sido usada durante más tiempo. Aproximación práctica del OPT basada en el principio de localidad temporal: si una página no se usó recientemente, probablemente no se usará pronto. Es el mejor algoritmo práctico, pero requiere soporte hardware.

Implementaciones de LRU:

1. Contadores hardware: la MMU guarda un timestamp de cada acceso.
Victima = página con el timestamp más antiguo.
Problema: requiere buscar el mínimo en toda la tabla ($O(N)$).
2. Lista doblemente enlazada: cada acceso mueve la página al final.
Victima = cabeza de la lista (la menos reciente).
Problema: actualizar la lista en cada acceso es muy costoso en CPU.
3. Aproximación por hardware: bit de referencia (R).
La MMU pone $R=1$ cuando accede a la página.
El SO lee y limpia R periódicamente.
Usado en la práctica por todos los SO modernos.

Con la cadena de referencia y 3 marcos:
Total LRU: 12 fallos (mejor que FIFO, peor que OPT).

3.4 Segunda Oportunidad (Second Chance / Clock)

Mejora del FIFO que evita reemplazar páginas recientemente usadas. Usa el bit de referencia R para dar una 'segunda oportunidad' antes de reemplazar.

Algoritmo de Segunda Oportunidad (implementación con cola circular):

Puntero → [Página A | R=0] → víctima: se reemplaza inmediatamente
Puntero → [Página B | R=1] → R ← 0, se avanza al siguiente
Puntero → [Página C | R=1] → R ← 0, se avanza al siguiente
Puntero → [Página D | R=0] → víctima: se reemplaza

Variante: Algoritmo del Reloj (Clock Algorithm)

El puntero recorre circularmente los marcos como agujas de reloj.

Si R=1: pone R=0 y avanza. Si R=0: reemplaza.

Muy eficiente: O(1) amortizado. Usado en Linux y BSD.

3.5 NRU — Not Recently Used (Clases de Páginas)

Clasifica las páginas en 4 clases según los bits R (referenciada) y M (modificada/dirty). El SO limpia periódicamente todos los bits R (en cada interrupción del reloj). Al necesitar reemplazar, se elige una página de la clase más baja disponible.

Clase	Bit R	Bit M	Descripción y política
0	0	0	No referenciada, no modificada. Mejor candidata: bajo coste, sin I/O de escritura.
1	0	1	No referenciada, modificada. Cuesta una escritura a disco pero no se usó recientemente.
2	1	0	Referenciada, no modificada. Activa recientemente; barata de reemplazar si no hay clase 0/1.
3	1	1	Referenciada y modificada. Peor candidata: activa y requiere escritura a disco.

NRU reemplaza aleatoriamente una página de la clase más baja que no esté vacía. Es fácil de implementar y aproxima razonablemente el LRU con un coste computacional mucho menor.

3.6 LFU y MFU — Frecuencia de Uso

Algoritmo	Descripción	Problema principal
LFU (Least Frequently Used)	Reemplaza la página con menor contador de referencias en un intervalo. Contador += bit_R en cada tick de reloj.	Una página muy usada en el pasado conserva un contador alto y permanece en memoria aunque ya no se use. Solución: envejecimiento (aging).

Algoritmo	Descripción	Problema principal
MFU (Most Frequently Used)	Reemplaza la página más frecuentemente usada. Razonamiento: la menos usada será más necesaria en el futuro.	Simulaciones demuestran que no supera a LRU en la práctica. Poco usado.
Aging (Envejecimiento)	Variante de LFU: en cada tick, el registro de N bits se desplaza a la derecha y se inserta el bit R en el bit más significativo.	Aproxima LRU con $O(1)$ por acceso y sin hardware especial. Usado en la práctica.

Ejemplo de Aging (registro de 8 bits por página, ticks cada 20 ms):

Página A: R=1 → 11000000 → tick → 11100000 → tick → 01110000
Página B: R=0 → 00000000 → tick → 00000000 → tick → 00000000
Página C: R=1 → 10000000 → tick → 11000000 → tick → 01100000

Víctima = Página B (registro = 00000000, menor valor).
Página A tiene el registro más alto → se usó más recientemente.

3.7 Comparativa de Algoritmos

Algoritmo	Fallos (*)	Implementación	Anomalía Bélády	Uso actual
OPT	9 (mínimo)	Imposible (requiere futuro)	No	Referencia teórica
LRU	12	Hardware costoso	No	Aproximado en SO
Clock/2ª opc.	~13	Muy eficiente $O(1)$	No	Linux, BSD, macOS
NRU	~14	Simple, $O(1)$	No	Embebidos, didáctico
Aging (LFU aprox)	~13	Simple, $O(1)$	No	Windows, variantes Linux
FIFO	15	Muy simple	Sí	Evitar en producción

(*) Número de fallos para la cadena de referencia del ejemplo con 3 marcos. Los valores varían con diferentes cargas de trabajo.

🔴 **Nota actualizada:** Linux utiliza una variante del algoritmo de reloj de dos manecillas (Two-Hand Clock) gestionado por el proceso kswapd. Mantiene dos límites: `páginas_libres_mínimo` y `páginas_libres_máximo`. Cuando las páginas libres caen por debajo del mínimo, kswapd despierta y libera páginas usando el algoritmo LRU-Approx con listas activa/inactiva. Las páginas se promueven a la lista activa al ser accedidas y se degradan a inactiva por el algoritmo de envejecimiento.

4. Asignación de Marcos: Políticas Local vs. Global

Cuando hay que reemplazar una página, el SO debe decidir de qué proceso tomar el marco. Esta decisión define la política de asignación de marcos.

Política	Descripción	Implicaciones
Local	El proceso solo puede reemplazar sus propias páginas. Cada proceso tiene un número fijo (o mínimo) de marcos.	Comportamiento predecible y aislado. Un proceso no puede degradar a los demás. Puede infrautilizar marcos.
Global	El proceso puede tomar marcos de cualquier otro proceso (elegido por el algoritmo de reemplazo).	Mayor utilización de la RAM total. Un proceso activo puede crecer a costa de otros. Comportamiento menos predecible.

La asignación mínima de marcos es crítica: cada proceso necesita un mínimo de marcos para poder ejecutar una instrucción sin entrar en thrashing. Por ejemplo, una instrucción que referencia dos operandos en memoria necesita al menos 3 marcos (código, operando 1, operando 2). Arquitecturas con modos de direccionamiento indirecto pueden necesitar más.

4.1 Algoritmos de Asignación de Marcos

Algoritmo	Descripción
Asignación igual	Divide los marcos disponibles equitativamente entre todos los procesos. Simple pero ignora las necesidades reales.
Asignación proporcional	Asigna marcos proporcionales al tamaño del proceso. Más justo pero no considera la actividad real.
Asignación por prioridad	Los procesos de mayor prioridad reciben más marcos. Al producirse un fallo, el proceso de baja prioridad pierde marcos.
Working Set (dinámico)	Asigna el número de marcos que necesita el working set del proceso en cada momento. Óptimo pero más complejo.

5. Thrashing y Modelo de Working Set

5.1 Thrashing

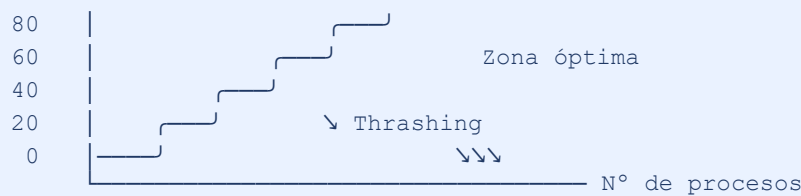
Si un proceso no tiene suficientes marcos para mantener sus páginas activas, produce fallos de página casi continuamente. El SO lo suspende para cargar la página faltante, pero al hacerlo, desaloja una página que pronto volverá a necesitar.

Thrashing: un proceso (o el sistema) está en thrashing cuando gasta más tiempo del CPU en transferencias de páginas (swap-in/out) que en realizar cómputo útil. La utilización del CPU cae drásticamente mientras la actividad de I/O de disco se dispara.

Utilización del CPU vs. Grado de multiprogramación:

CPU % |
100 |





↑
Punto de thrashing: añadir más procesos reduce la utilización del CPU en lugar de aumentarla.

El thrashing ocurre porque el SO usa un algoritmo de reemplazo global: al aumentar el grado de multiprogramación, cada proceso recibe menos marcos. Cuando los marcos son insuficientes para el working set de cada proceso, se produce un ciclo de fallos continuos.

5.2 Modelo de Working Set

El working set $W(t, \Delta)$ de un proceso en el instante t es el conjunto de páginas referenciadas en las últimas Δ unidades de tiempo (ventana del working set). Es la aproximación más precisa de las páginas que el proceso necesita activamente.

Cadena de referencia: ... 2, 6, 1, 5, 7, 7, 1, 1, 2, 6, 1 ...

instante actual t

Ventana $\Delta = 10$ referencias hacia atrás:

Referencias: 2, 6, 1, 5, 7, 7, 1, 1, 2, 6

Working Set $W(t, 10) = \{1, 2, 5, 6, 7\} \rightarrow$ necesita 5 marcos

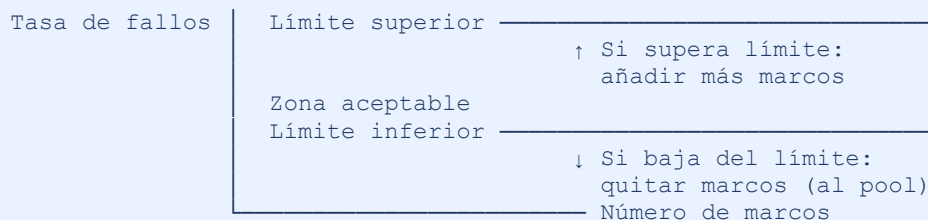
Si el proceso solo tiene 3 marcos $\rightarrow$ thrashing inevitable.

Si tiene ≥ 5 marcos $\rightarrow$ ejecución eficiente.

La estrategia de working set para prevenir thrashing: el SO solo carga un proceso en memoria si hay suficientes marcos disponibles para su working set completo. Si la suma de los working sets supera los marcos disponibles, algún proceso se suspende (swap-out completo) hasta que haya espacio.

5.3 Control de la Tasa de Fallos de Página (PFF)

Una alternativa al working set es controlar directamente la frecuencia de fallos de página (Page Fault Frequency — PFF):



Si ningún proceso puede ceder marcos y los fallos siguen:
 $\rightarrow$ Suspende (swap-out) algún proceso para liberar marcos.

● **Atención:** Señales de thrashing en un sistema en producción: alta actividad de swap (vmstat 1: si 'si' y 'so' son altos constantemente), utilización del CPU baja (~20–40%) mientras el sistema parece 'congelado', tiempo de carga de procesos muy alto.

Herramientas de diagnóstico: vmstat, sar, perf stat, /proc/vmstat (pgpgin, pgpgout, pswpin, pswpout).

6. Overlays: Carga Dinámica Manual (Contexto Histórico)

Antes de que la memoria virtual fuera una característica del SO, los programadores debían gestionar manualmente qué partes del programa estaban en memoria. Esta técnica se denominaba overlay (sobreposición).

Un overlay consiste en dividir manualmente el programa en módulos (overlays), cargando en memoria solo los necesarios en cada momento. Cuando se necesita un módulo no residente, se carga sobre el espacio de otro que ya no se necesita. El programa incluye un 'overlay manager' que gestiona esta carga.

Programa de 256 KB con RAM disponible de 64 KB:

```
[Módulo principal: 32 KB] ← siempre en memoria
[Overlay A: 48 KB] ← fase 1 (procesamiento de datos)
[Overlay B: 48 KB] ← fase 2 (generación de informe) } se solapan
[Overlay C: 48 KB] ← fase 3 (comunicación de red)   } en la misma
                                                    } región RAM
```

Organización en RAM:

Módulo principal (32 KB, siempre)	Región overlay (A, B o C según el momento de ejecución: 48 KB)
--------------------------------------	---

Total RAM usada: $32 + 48 = 80$ KB ← cabe en 64 KB con gestión cuidadosa.

La dificultad del overlay es que el programador debe diseñar explícitamente qué módulos pueden coexistir en memoria y cuáles se excluyen mutuamente. Esto es complejo, propenso a errores y limita la mantenibilidad del código.

Aspecto	Overlays	Memoria Virtual (Paginación)
Responsabilidad	Del programador (manual).	Del SO + hardware (transparente).
Transparencia	No: el programador debe conocer la RAM.	Total: el proceso no sabe que hay paginación.
Granularidad	Por módulos (KB a MB).	Por páginas (4 KB típico).
Uso actual	Obsoleto (excepto en muy pequeños microcontroladores).	Universal en todos los SO modernos.

🔴 **Nota actualizada:** Los overlays son prácticamente obsoletos en sistemas de uso general. Sin embargo, en microcontroladores con muy poca flash/RAM (< 32 KB) como los PIC o AVR de 8 bits, variantes de esta técnica siguen usándose. En sistemas embebidos modernos con más recursos, incluso los microcontroladores ARM Cortex-M implementan XIP (Execute In Place) sobre flash como alternativa más eficiente.

7. Memoria Virtual en Sistemas Operativos Actuales

7.1 Linux: Gestión de Memoria Virtual

Componente Mecanismo	Descripción
Demand paging	Páginas cargadas solo cuando se acceden por primera vez. El kernel mapea regiones pero no carga físicamente.
Copy-on-Write (COW)	fork() comparte páginas del padre en modo lectura. Al primer intento de escritura del hijo → page fault → se copia solo esa página.
kswapd / kcompactd	Procesos del kernel que mantienen suficientes páginas libres. kswapd ejecuta el algoritmo LRU-Approx. kcompactd desfragmenta la memoria física.
Listas activa/inactiva	El kernel mantiene dos listas LRU: active_list (usadas recientemente) e inactive_list (candidatas a swap). Las páginas se 'envejecen' moviéndose entre listas.
OOM Killer	Cuando la memoria se agota completamente (sin swap disponible), el OOM Killer elige y termina un proceso para liberar RAM. El proceso se elige por una puntuación basada en tamaño, prioridad y tiempo de vida.
mmap y lazy allocation	mmap() reserva espacio virtual pero no asigna páginas físicas hasta el primer acceso. Esto permite reservar grandes regiones sin consumir RAM de inmediato.
Transparent Huge Pages (THP)	El kernel promueve automáticamente páginas de 4 KB contiguas a páginas de 2 MB (huge pages) para reducir el tamaño de la tabla de páginas y mejorar el TLB hit rate en cargas con grandes datasets.
NUMA awareness	En sistemas con múltiples nodos NUMA, el kernel asigna preferentemente páginas del nodo local al proceso para minimizar la latencia de acceso a memoria.

7.2 Área de Swap en Linux

Cuando el sistema se queda sin RAM libre, las páginas inactivas se mueven al área de swap (partición o archivo de paginación en disco). En Linux pueden coexistir múltiples áreas de swap con diferentes prioridades.

```
# Ver estado del swap:
free -h
swapon --show
cat /proc/swaps

# Ver actividad de paginación en tiempo real:
vmstat 1          # si = swap in (páginas/s), so = swap out (páginas/s)
sar -B 1          # pgpgin/s, pgpgout/s, fault/s, majflt/s

# Ver working set aproximado de un proceso (PID = 1234):
cat /proc/1234/status | grep -i vm
# VmRSS = páginas en RAM | VmSwap = páginas en swap
```

```
# Ver estadísticas de la memoria virtual del kernel:
cat /proc/vmstat | grep -E 'pgfault|pgmajfault|pswpin|pswpout'
# pgfault      = fallos de página totales (minor + major)
# pgmajfault   = fallos de página mayores (requirieron acceso a disco)
```

7.3 Windows: Virtual Memory Manager (VMM)

Característica	Descripción
Espacio virtual	4 GB en x86-32 (2 GB usuario + 2 GB kernel). 128 TB en x86-64.
Archivo de paginación	pagefile.sys: uno o más archivos en disco. Tamaño configurable (mínimo, máximo) o automático.
Working Set Trimming	El VMM reduce dinámicamente el working set de los procesos en segundo plano cuando la RAM es escasa.
Balanced Set Manager	Proceso del sistema que equilibra los working sets para maximizar el rendimiento global.
Standby/Modified Lists	Páginas 'libres' se mantienen en la Standby List (limpias) o Modified List (sucias). Pueden reutilizarse rápidamente si el mismo proceso las necesita (soft fault).
SuperFetch	Analiza el historial de uso para precargar anticipadamente las páginas más usadas al inicio del sistema (prepaging inteligente).

7.4 Comparativa de Gestión de Memoria Virtual en SO

SO	Algoritmo reemplazo	Swap	Huge Pages	Característica destacada
Linux	LRU-Approx (Clock de 2 manos)	Partición o archivo	THP automático	OOM Killer, NUMA, cgroups
Windows	LRU-Approx (Balanced Set)	pagefile.sys	No automático	SuperFetch, Standby List
macOS	LRU con compresión (Compressor)	Compresión primero	No	Compressed Memory antes de swap
Android	LRU + zswap + OOM Killer	zRAM (compresión)	No	Low Memory Killer (LMK)

📌 **Nota actualizada:** macOS y iOS usan Memory Compressor como primera línea de defensa antes del swap: las páginas inactivas se comprimen en RAM (ratio ~2:1), duplicando efectivamente la RAM disponible. Solo si la RAM comprimida también se llena, macOS hace swap a disco. En iOS no hay swap a disco por limitaciones del flash NAND; si la RAM se agota, las apps en segundo plano son terminadas directamente.

8. Estudio Independiente

1. Un proceso genera la cadena de referencia: 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6. Calcule el número de fallos de página con 3 y 4 marcos para los algoritmos FIFO, LRU y OPT. ¿Se observa la anomalía de Bélády con FIFO?
1. Explique el principio de localidad temporal y espacial. ¿Por qué sin este principio la memoria virtual sería inviable en la práctica?
2. ¿Qué es el thrashing? Describa cómo el modelo de Working Set lo detecta y previene. ¿Qué hace el SO cuando detecta que la suma de los working sets supera los marcos disponibles?
3. ¿Qué hace el OOM Killer de Linux? ¿Cómo selecciona el proceso víctima? ¿Es posible configurar qué procesos son inmunes al OOM Killer? ¿En qué situaciones es preferible al thrashing indefinido?
4. Investigue el mecanismo Copy-on-Write (COW) en Linux. ¿Cómo interactúa con fork()? ¿Qué ocurre a nivel de tabla de páginas cuando el proceso hijo intenta escribir en una página compartida con el padre?

— Fin de la Conferencia 7 —